

<!-- Course: Pi Agent - Crash Course |
Minimal Coding Agent -->

<!-- Source: VideoTranscriber_Pi Agent - Crash Course _ Minimal Coding Agent.txt -->

-- Page 1: Pi Agent Harness Overview

Header

🕒 ~4 min | Section 1 | 🟢 Easy

What You'll Learn

See why Pi is intentionally small, what it leaves out on purpose, and how that design makes it easier to extend than a feature-heavy coding harness.

Deep Dive Explanation

- Pi starts as a minimal harness with only a handful of built-in tools, so the default surface area stays small and understandable.
- It does not assume you want MCP, sub-agents, permission pop-ups, plan mode, or built-in to-dos; instead, those behaviours are added only when you need them.
- The key idea is that Pi is not trying to be complete at install time. It is trying to stay configurable so the harness can grow with your workflow.
- That means Pi behaves more like a foundation layer than a full product bundle: install first, then shape the harness around your coding style.

Visual Summary

,

Minimal default install

-
- 4 built-in tools
- no MCP by default
- no sub-agents by default
- no permissions UI by default
- no plan/to-do system by default

Add only what you need later.

...

Real-World Use First

Scenario: You want a coding assistant that stays small on day one but can later support custom automation, safety checks, and local workflows.

Why it matters: Pi avoids forcing opinionated features on you. That keeps the initial setup light and makes advanced behaviour something you opt into instead of inherit by default.

Process Flow / Steps

1. Install Pi.
2. Start with the small built-in toolset.
3. Add features only when a real workflow needs them.
4. Keep the harness shaped around your use case.

Key Concepts

- **Minimal harness:** A small base system that starts simple and only grows when configured.
- **Extensibility first:** New behaviour is added by configuration, prompts, or extensions rather than hard-coded defaults.
- **No-opinion startup:** Pi does not force MCP, sub-agents, or plan mode on every user.

Try This Right Now

- List the default tools in your Pi install and compare them with a more feature-heavy harness.
- Write down one capability you would add only if a real project needs it.

Flashcards

Q	A
---	---
What is Pi designed to be at the start?	A minimal agent harness that stays small by default
What does Pi intentionally leave out initially?	MCP, sub-agents, permission pop-ups, plan mode, and built-in to-dos
What is the main design goal?	Let the harness grow with your workflow

One-Page Revision

- Pi starts minimal, not feature-heavy.
- Optional capabilities are added only when needed.
- The harness is a base layer you shape over time.
- Simplicity is a design choice, not a limitation.



-- Page 2: Login, Models, and Scopes

Header

🕒 ~5 min | Section 2 | 📖 Medium

What You'll Learn

Understand how Pi becomes useful only after login, how model providers are attached, and how scoped models let you switch quickly between your preferred options.

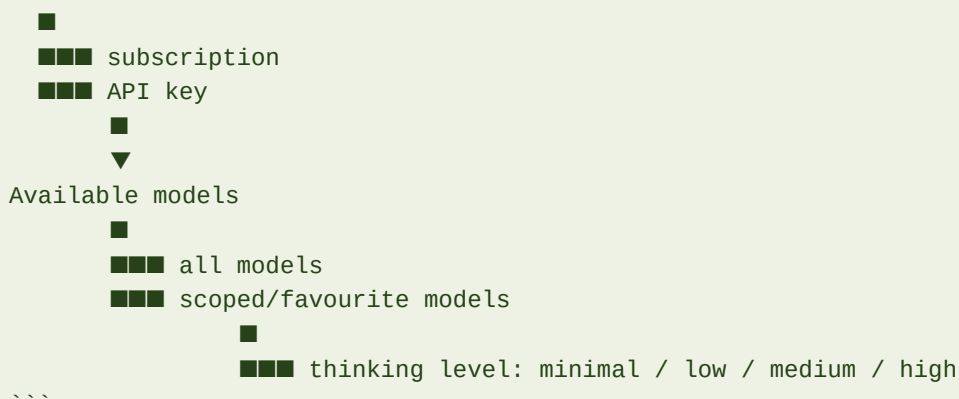
Deep Dive Explanation

- Pi installs without models, so the first real setup step is authentication through either a subscription or an API key.
- Once logged in, Pi can expose model choices from multiple providers in the same interface.
- The model selector is not just a static list; it separates all available models from your scoped or favourite models.
- Scoped models are the practical shortcut: they let you mark the models you reach for most often so switching stays quick during daily work.
- Thinking level is another layer of control. Some models expose minimal, low, medium, and high thinking modes, so you can tune depth versus speed without changing tools.

Visual Summary

,

Login



Real-World Use First

Scenario: You use Hugging Face for broad model access, but you keep three preferred models scoped because you switch between them every day.

Why it matters: The model menu becomes a workflow tool, not just a settings screen. You spend less time searching and more time working with the model best suited to the task.

Process Flow / Steps

1. Install Pi.
2. Log in with a subscription or API key.
3. Open the model chooser.
4. Mark your preferred models as scoped.
5. Use thinking controls when a task needs more or less reasoning depth.

Key Concepts

- **Login:** The step that activates providers and makes models available.
- **Scoped models:** A curated shortlist of models you use often.
- **Thinking level:** A reasoning-depth control exposed by some models.

Try This Right Now

- Add one model to your scoped list and one model you rarely use.
- Switch between them once to see how much faster the scoped workflow feels.

Flashcards

Q	A
---	---
Why do you need to log in after installing Pi?	Because the install does not include any models by default
What are scoped models?	The models you mark as favourites for quick switching
What does thinking level change?	How much reasoning depth the selected model uses

One-Page Revision

- Pi needs login before models appear.
- You can use subscriptions or API keys.

- Scoped models are a fast path to your favourites.
- Thinking levels let you trade speed for depth.



-- Page 3: Prompt Templates

Header

⏱ ~4 min | Section 3 | 🟢 Easy

What You'll Learn

See how Pi turns repeatable prompts into reusable commands so you do not keep retyping the same instruction over and over.

Deep Dive Explanation

- Prompt templates behave like custom slash commands: typing the template name expands into a larger prompt.
- Pi can create these prompts for you, which matters because it can read its own docs and generate a template that fits the current workspace.
- The main value is consistency. If you repeatedly ask for code review, bug triage, or refactoring help, the prompt can encode that workflow once.
- Templates can include variables, so the same command can adapt to different files or tasks.
- Because templates live in the Pi configuration directory, they survive across sessions and become part of your personal harness setup.

Visual Summary

,

Type short command

```
graph TD; A[ ] --> B[Template expands]; B --> C[ ]; C --> D[Detailed prompt runs]; D --> E[ ]; E --> F[Same workflow every time]; F --> G[...];
```

Real-World Use First

Scenario: You review code every day and always want the same checklist: correctness, readability, security, and test coverage.

Why it matters: A prompt template removes repeated typing and makes the review quality more consistent because every review starts from the same checklist.

Process Flow / Steps

1. Ask Pi to create a custom prompt.
2. Save the prompt in the configuration folder.
3. Reload Pi after editing prompts.
4. Call the prompt like a slash command.

Key Concepts

- **Prompt template:** A reusable prompt that expands from a short command.
- **Variables:** Placeholders that let one prompt adapt to multiple tasks.
- **Reload:** The step required after prompts change so Pi sees the update.

Try This Right Now

- Create one template for code review and one for refactoring.
- Reload Pi and verify both appear as usable commands.

Flashcards

Q	A
---	---
What is a prompt template in Pi?	A reusable prompt that acts like a custom slash command
Why are prompt templates useful?	They avoid retyping the same workflow instructions
What usually happens after editing prompts?	Pi must be reloaded so the new prompt appears

One-Page Revision

- Prompt templates are reusable slash-style commands.
- Pi can generate templates from your request.
- Variables make one template work in multiple situations.
- Reload after changes so Pi loads the update.



-- Page 4: Skills and Command Execution

Header

🕒 ~5 min | Section 4 | 📖 Medium

What You'll Learn

Understand how Pi discovers skills, how those skills can be shared across home and workspace folders, and how command execution works inside the terminal session.

Deep Dive Explanation

- Pi reads skills from both the home `.agents` directory and the workspace `.agents` directory, so the same capability can be shared globally or kept local.
- It also loads skills from the `.clote` directory, which means Pi can pick up an existing skill ecosystem instead of forcing a new one.
- Skills appear in the slash skill picker, where you can run them directly from the Pi interface.
- Pi also supports terminal commands with a single `!` prefix, and commands prefixed with `!!` can avoid polluting conversation history.
- That distinction matters when you want to run a command without turning the terminal command itself into part of the agent context.

Visual Summary

,

Skill sources

```
■■■ home .agents
■■■ workspace .agents
■■■ .clote
■■■ Pi-specific skills folder
  ■
  ▼
  slash skill picker
    ■
    ■■■ ! command / !! command
  ...
```

Real-World Use First

Scenario: You keep a shared skill in your home directory and a workspace-specific skill for one repo.

Why it matters: Pi can load both without extra wiring, so reusable behavior stays global while project-specific behavior stays local.

Process Flow / Steps

1. Put a skill in .agents or the Pi skills folder.
2. Open Pi and check the skill list.
3. Run a skill from the slash picker.
4. Use ! for shell commands and !! when you want to keep the command out of context.

Key Concepts

- **Skill discovery:** Pi scans multiple directories for skills automatically.
- **Workspace-local skill:** A skill that belongs to one repository or project only.
- **History-aware command:** A shell command that becomes part of the conversation context unless you use !!.

Try This Right Now

- Move one skill from the workspace to the home directory and confirm Pi still detects it.
- Run one harmless shell command with ! and one with !! to see the difference in history.

Flashcards

Q	A
---	---
Where does Pi look for skills?	In home and workspace <code>`.agents`</code> , the <code>`.clote`</code> directory, and Pi-specific skill folders
What does <code>`.`</code> do in Pi?	Runs a shell command and records it in conversation history
What does <code>`.`</code> do?	Runs a shell command without adding it to the conversation context

One-Page Revision

- Pi discovers skills from several locations.
- Skills can be shared globally or kept workspace-local.
- ! records shell commands; !! avoids context clutter.
- The skill system is modular and easy to extend.



-- Page 5: Themes and UI Customization

Header

⌚ ~4 min | Section 5 | 🟢 Easy

What You'll Learn

See how Pi lets you change the terminal experience itself, from built-in themes to custom workspace-specific appearance settings.

Deep Dive Explanation

- Pi is not only customizable in behavior; its terminal UI can be changed too.
- The built-in themes are simple, but you can ask Pi to create a new theme for the current workspace.
- Workspace-scoped themes matter because they let you experiment without affecting your global Pi setup.
- After creating or changing a theme, Pi needs a reload before the new theme appears in the settings UI.
- The point is that UI treatment is part of the harness, not an afterthought. Pi lets you shape how the agent feels while you use it.

Visual Summary

,

Settings

```
■
■■■ built-in theme
■■■ custom workspace theme
  ▼
  Reload Pi
  ▼
  New theme appears
  ...
```

Real-World Use First

Scenario: You want a playful workspace theme for experimentation but prefer your standard dark theme in everything else.

Why it matters: Workspace-scoped UI changes keep your working environment tailored without disrupting the rest of your setup.

Process Flow / Steps

1. Open settings.
2. Ask Pi to generate a workspace theme.
3. Reload Pi.
4. Select the new theme.

Key Concepts

- **Workspace theme:** A theme stored for one repository or working area only.
- **Reload:** Required after theme updates so the UI refreshes.
- **UI customization:** Pi allows the terminal interface to be shaped to your taste.

Try This Right Now

- Create one new theme for your current workspace and compare it against the built-in dark theme.
- Check whether the theme color choices make thinking state easier to read.

Flashcards

Q	A
---	---
What are the built-in Pi themes?	Dark and light
Why create a workspace-specific theme?	So the look changes only for that project or folder
What must you do after changing the theme?	Reload Pi

One-Page Revision

- Pi supports built-in and custom themes.
- Workspace themes keep experimentation isolated.
- Reload after changes to make them visible.
- UI customization is part of the harness design.



-- Page 6: Context Files

Header

🕒 ~5 min | Section 6 | 📖 Medium

What You'll Learn

Understand how Pi composes context from standard agent instruction files and why home-directory instructions deserve extra care.

Deep Dive Explanation

- Pi follows the agents.md pattern and also reads clod.md, so it can merge instructions from multiple context files automatically.
- Workspace-level files are useful for project-specific instructions, while home-directory files act like defaults across all projects.
- Pi loads both home and workspace versions when they exist, which means a file in your home directory can quietly influence many sessions.
- That makes home context files powerful but risky: if they are too broad or stale, they affect more conversations than you might expect.
- The safest strategy is to keep workspace-specific instruction files precise and use home files only for genuinely universal preferences.

Visual Summary

,

Context files Pi reads

```
■■■ workspace agents.md
■■■ workspace clod.md
■■■ home .agents / agents.md
■■■ home .clote / clod.md
```



merged into context

...

Real-World Use First

Scenario: Your workspace says, "be concise and prioritize tests," while your home file says, "always include deployment notes." Pi reads both and combines them.

Why it matters: Context files are how you encode persistent preferences. If they are too broad, they can shape answers in places you never intended.

Process Flow / Steps

1. Put project rules in workspace context files.
2. Put broad defaults in home context files.
3. Open Pi in that workspace.
4. Let Pi merge both sources into context.

Key Concepts

- **Workspace context:** Instructions that apply only to one repository or folder.
- **Home context:** Global instructions that can influence many projects.
- **Merged context:** The combined instruction set Pi loads at runtime.

Try This Right Now

- Compare your home instruction files with one workspace instruction file.
- Identify one rule that should remain global and one that should be project-specific.

Flashcards

Q	A
---	---
Which instruction files does Pi read?	`agents.md` and `clod.md` from both workspace and home locations
Why are home context files important?	They affect many sessions, not just one project
What is the safest strategy for instruction files?	Keep workspace files precise and home files only for universal preferences

One-Page Revision

- Pi reads both workspace and home context files.
- agents.md and clod.md are merged automatically.
- Home files have broad influence, so keep them carefully curated.
- Workspace files are better for project-specific rules.



-- Page 7: Extensions

Header

⏱ ~6 min | Section 7 | 📖 Hard

What You'll Learn

See how Pi extensions add new behavior such as welcome messages, confirmation guards, and other workflow changes that are not built in by default.

Deep Dive Explanation

- Extensions are Pi's way to add missing features after installation without changing the core harness.
- A workspace extension can do small things like display a welcome quote at startup, which proves the system can modify the UI and startup behavior.
- A more important extension pattern is the command guard: Pi can inspect a shell command and ask for confirmation before dangerous actions like `rm -rf` or `git push --force`.
- The demo shows that this guard is not theoretical - without the extension, a destructive command can run immediately; with it, Pi pauses and asks for permission.
- Extensions therefore act like safety rails and workflow augmenters. They turn Pi into a custom harness rather than a fixed product.

Visual Summary

,

Extension

```
■
■■■ startup quote
■■■ danger command guard
■■■ UI behavior changes
■■■ workspace-specific logic
...`
```

Real-World Use First

Scenario: You use Pi for coding and never want a typo to destroy a folder or push force to a shared branch without confirmation.

Why it matters: The extension turns a dangerous default into a controlled workflow that protects your repo from accidental damage.

Process Flow / Steps

1. Ask Pi to create a workspace extension.
2. Reload Pi.
3. Trigger the feature the extension controls.
4. Confirm the extension actually intercepts the workflow.

Key Concepts

- **Extension:** Code that adds new behavior to Pi after installation.
- **Danger command guard:** A safety layer that asks for confirmation before destructive commands run.
- **Workspace extension:** An extension that affects only one project.

Try This Right Now

- Build a guard for one destructive command you never want to run accidentally.
- Test that Pi stops and asks before the command executes.

Flashcards

Q	A
---	---
What is the purpose of a Pi extension?	To add new behaviour that is not built in by default
What does the danger command guard protect against?	Accidental destructive commands like `rm -rf` or `git push --force`
Why reload Pi after adding an extension?	So the new extension is picked up and activated

One-Page Revision

- Extensions add behavior after installation.
- They can modify startup, UI, or command handling.
- A guard extension can stop destructive shell commands.
- Workspace extensions keep custom behavior local.



Header

⏱ ~4 min | Section 8 | 🟢 Easy

What You'll Learn

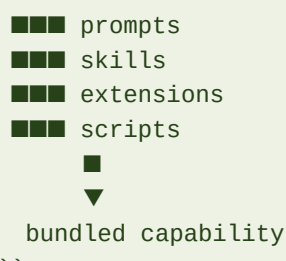
See how packages bundle prompts, skills, and extensions together so you can add larger capability blocks in one step.

Deep Dive Explanation

- Packages are bundles of functionality contributed by the community: prompts, skills, extensions, and other related pieces packaged together.
- They act like a curated install path for a capability family. Instead of assembling every piece manually, you can install a package that already groups them.
- Because packages may contain scripts, you should treat them like any other code dependency: inspect what they do before trusting them.
- The package model is useful for things like MCP adapters, web search support, or sub-agent behaviour that Pi does not include by default.
- In other words, packages are Pi's ecosystem layer: community-built modules that extend the minimalist core.

Visual Summary

Package



Real-World Use First

Scenario: You want web search, MCP support, and sub-agent helpers without wiring each one by hand.

Why it matters: A package lets you install a functional capability set in one move instead of assembling each piece from scratch.

Process Flow / Steps

1. Browse available packages.
2. Review the package contents.
3. Install only the ones you trust.
4. Let the bundle add the related prompts, skills, or extensions.

Key Concepts

- **Package:** A bundle of related Pi capabilities.
- **Curated install:** A faster path to a feature set than manual assembly.
- **Trust boundary:** The need to inspect package scripts before running them.

Try This Right Now

- Pick one package and inspect what it installs before using it.
- Compare the effort required to install it versus wiring the same capability manually.

Flashcards

Q	A
---	---
What is a package in Pi?	A bundle of prompts, skills, extensions, and related functionality
Why are packages useful?	They let you add a capability set in one step
What should you check before installing one?	Its scripts and contents, because packages may run code

One-Page Revision

- Packages bundle multiple Pi capabilities.
- They speed up setup for common workflows.
- Inspect scripts before trusting any package.
- Packages represent Pi's community ecosystem.



-- Page 9: Sessions and Thread Management

Header

⏱ ~6 min | Section 9 | 📖 Hard

What You'll Learn

Understand how Pi organizes sessions, supports branching and export, and lets you recover from a bad turn without losing the whole conversation.

Deep Dive Explanation

- Sessions are one of Pi's strongest features because they make thread navigation and recovery explicit instead of hidden.
- You can rename a thread with slash name, inspect its location with slash session, and see where the JSONL history lives on disk.
- The session tree lets you move backward and forward through a conversation so you can recover from a mistaken branch without starting over.
- fork and clone give you two different branching strategies: fork creates a separate working thread; clone duplicates the current position within the existing session tree.
- Session export matters because it turns a live chat into something you can archive, inspect, or reuse for training a model or building a new skill.

Visual Summary

`

Current session

```
■■■ rename
■■■ tree navigation
■■■ fork
■■■ clone
■■■ compact
■■■ export JSONL
...`
```

Real-World Use First

Scenario: You try one change in a conversation, realize it was the wrong direction, and want to resume from the last good assistant turn.

Why it matters: Sessions let you recover, branch, and export without losing the useful part of the conversation.

Process Flow / Steps

1. Name the session.
2. Inspect its path and JSONL location.
3. Use tree navigation to move around the conversation.
4. Fork or clone when you need a parallel branch.
5. Export the session when you want to archive or reuse it.

Key Concepts

- **Session:** A saved conversation thread with its own history and directory.
- **Tree navigation:** The ability to move backward and forward through message history.
- **Fork:** Creates a separate continuation branch.
- **Clone:** Duplicates the current session position.
- **Export:** Produces a file you can reuse outside Pi.

Try This Right Now

- Rename one session and then locate it on disk.
- Move backward in the tree, then fork it and compare the resulting branches.

Flashcards

Q	A
---	---
What does `slash name` do?	Renames the current session thread
What is the purpose of session tree navigation?	To move backward and forward through conversation history
What is the difference between fork and clone?	Fork creates a separate branch; clone duplicates the current position
Why export a session?	To archive, inspect, or reuse the conversation outside Pi

One-Page Revision

- Sessions make Pi threads recoverable and inspectable.
- Name, navigate, fork, clone, compact, and export.

- JSONL export can be reused for analysis or training.
- Sessions are one of Pi's most practical power features.